Assignment 3 solution

1. 【考点：内存管理的基本概念】在现代计算机操作系统的内存管理机制中，关于地址空间划分与内存保护，下列描述绝对准确的是（ ）

   A. 任何用户进程在运行期间发出的物理地址，都会被硬件 MMU（内存管理单元）自动拦截并与限长寄存器进行比对，以实现严格的越界保护
   B. 在基于重定位寄存器和界地址寄存器（限长寄存器）的机制中，界地址寄存器内部存放的是该进程在物理内存中的最大绝对物理地址
   C. 在页式存储管理系统中，页表项内的“有效/无效（状态位）”字段主要用于实现内存共享，而非内存保护
   D. 段式内存管理相比纯页式管理天然更容易实现共享和保护，因为段是逻辑上有意义的完整信息集合，操作系统可以针对整个段赋予统一的读、写或执行权限

   【答案】：D
   【解析】：
   A 选项错误：用户进程在代码中发出的是逻辑地址（虚拟地址），而不是物理地址。MMU 的作用正是将逻辑地址转换为物理地址并进行越界检查
   B 选项错误：界地址寄存器（限长寄存器）存放的是进程逻辑地址空间的最大长度，而不是物理绝对地址；重定位寄存器存放的才是物理首地址
   C 选项错误：页表项内的“有效/无效”位主要用于判断该页是否已调入物理内存（请求页式中的缺页异常机制）以及访问权限保护，不用于共享判断
   D 选项正确：段是信息的逻辑单位（如代码段、数据段），其内部属性一致，因此直接对段表项设置保护位和共享标志比对无逻辑意义的“页”设置要合理和方便得多

2. 【考点：连续分配管理方式】在采用动态分区分配（可变分区分配）管理物理内存的系统中，关于各种分区分配算法及其引发的碎片问题，下列说法正确的是（ ）
   A. 无论采用首次适应（First Fit）还是最佳适应（Best Fit）算法，动态分区分配的过程必然会在系统内存中产生大量的内部碎片
   B. 邻近适应算法（Next Fit）每次分配时都会从内存低地址端开始扫描，这通常能比首次适应算法更快地找到合适的空闲块
   C. 最佳适应算法（Best Fit）的底层实现需要将所有空闲分区按大小递减次序排列，这导致其分配和回收时维护数据结构的开销极大
   D. 若采用紧凑（Compaction）技术来消除动态分区分配产生的外部碎片，系统必须暂停当前正在运行的、即将被移动的用户进程，并动态修改其重定位寄存器的值

   【答案】：D
   【解析】：
   A 选项错误：动态分区分配的特点就是按照进程实际所需大小进行分配，因此不存在内部碎片，只会产生外部碎片。产生内部碎片的是固定分区分配和页式分配
   B 选项错误：邻近适应算法是从上次查找结束的位置继续向后扫描，而不是每次从低地址开始（这是首次适应的做法）
   C 选项错误：最佳适应算法要求空闲分区按大小递增排序，以便最先找到能满足要求且最小的空闲块
   D 选项正确：紧凑技术会移动内存中程序的位置，因此程序必须是可动态重定位的，在移动时需暂停运行，并在移动后更新基址寄存器

3. 【考点：页式管理与段式管理的对比】深入对比基本页式存储管理和基本段式存储管理，以下论述中完全正确的一项是（ ）

A. 从物理内存的利用率来看，段式管理由于采用了离散分配思想，不仅消除了外部碎片，也彻底根除了内部碎片
B. 在纯页式系统中，逻辑地址的结构对用户是透明的；而在段式系统中，逻辑地址结构的划分显式可见，程序员编程时可直接控制段的划分
C. 为了结合二者的优势，段页式管理系统为每个进程建立一张全局段表，并在每个段表项中直接连续存放该段所包含的所有物理页框号
D. 页式管理中逻辑地址到物理地址的数学映射计算，完全由操作系统内核的代码在每次访存时动态软件求解完成，无需底层硬件介入

【答案】：B
【解析】：
A 选项错误：段式管理中段与段之间是不连续分配的，由于段长不固定，会产生外部碎片（不存在内部碎片）
C 选项错误：段页式管理的段表项中存放的是该段对应的页表首地址和页表长度，而不是直接存放物理页框号。地址映射是：段表 -> 页表 -> 物理页
D 选项错误：地址转换频率极高，完全由操作系统软件计算会导致系统瘫痪，地址转换必须由 CPU 内部的 MMU 硬件完成
B 选项正确：分页是系统行为，页面大小由硬件决定，逻辑地址是一维的（对用户透明）；分段是用户行为，逻辑地址是二维的（段号+段内偏移）

4. 【考点：页式管理】在具有快表（TLB）的单级页表分页存储管理系统中，逻辑地址转换和访存性能的计算至关重要。若 TLB 的命中率为 p，访问一次物理内存的时间为 t，访问一次 TLB 硬件的时间为 ϵ。在不考虑缺页异常的前提下，该系统的有效访存时间（EAT）理论表达式是（ ）

A. $p(t + \epsilon) + (1 - p)(t + \epsilon)$
B. $p \cdot \epsilon + (1 - p)(t + \epsilon)$
C. $p(t + \epsilon) + (1 - p)(2t + \epsilon)$
D. $p(2t + \epsilon) + (1 - p)(3t + \epsilon)$

【答案】：C
【解析】：
命中 TLB 时： 花费时间 ϵ 查找 TLB 获取物理块号，再花费时间 t 访问物理内存获取数据。耗时：$(\epsilon + t)$。概率为 p
未命中 TLB 时： 花费时间 ϵ 查找 TLB 发现未命中，再去物理内存中查找页表花费时间 t，获得物理块号后（同时硬件自动更新 TLB），最后再次花费时间 t 访问物理内存获取真实数据。耗时：$(\epsilon + t + t) = (\epsilon + 2t)$。概率为 $(1 - p)$
综合有效访存时间：$EAT = p(t + \epsilon) + (1 - p)(2t + \epsilon)$。选项 C 正确

5. 【考点：虚拟内存基本概念】虚拟存储器使得大程序能够在小内存中运行，其基础是局部性原理。下列关于虚拟内存概念与局部性原理的表述中，存在明显错误的是（ ）

A. 虚拟存储器的最大支持容量受到计算机 CPU 地址总线位数（如 32 位系统的 4GB 寻址空间）以及底层物理硬件（内外存容量之和）的共同制约
B. 空间局部性原理表明，如果程序中的某条指令被执行，则该指令的周围指令在不久的将来极有可能被执行（如数组的顺序遍历）
C. 虚拟存储机制不仅允许进程的逻辑地址空间大于系统实际物理内存，还支持将一个进程的代码和数据作为一个巨大的连续整体直接映射到外存的连续物理扇区中运行
D. 虚拟存储技术的核心特征之一是“多次性”，即允许一个作业在运行过程中，被分多次调入内存，而不是一次性全部装入

【答案】：C
【解析】：
C 选项说法错误。虚拟存储器的核心实现必须依赖于离散分配机制（如请求分页或请求分段）。如果像连续分配那样要求程序作为一个“巨大的连续整体”运行，由于物理内存不足以装下整体，根本无法实现虚拟内存。虚拟内存是将逻辑空间切片，只把当前需要的部分离散地放入内存。其他选项均为标准正确概念

6. 【考点：请求页式管理】当 CPU 在执行指令的过程中，发现所需的页面不在物理内存中，便会引发缺页异常（Page Fault）。关于该异常的处理流转机制，下列描述正确的是（ ）

A. 缺页异常属于硬件触发的外部异步中断，因此 CPU 必须等待当前这条机器指令完全执行结束后，才会在指令周期的末尾响应并处理它
B. 在一条极其复杂的机器指令（如跨页面访问操作数）的执行期间，CPU 最多只可能触发并处理一次缺页中断
C. 当系统处理缺页异常时，如果物理内存中没有空闲页框，需使用页面置换算法淘汰一页；若被选中的淘汰页面其“修改位（Dirty Bit）”为 0，则无需将其内容重新写回磁盘
D. 处理缺页异常的代码逻辑被固化在 CPU 内部的微程序中，属于纯硬件动作，无需切换到操作系统内核态执行

【答案】：C
【解析】：
A 选项错误：缺页中断属于内部异常（故障 Fault），它是在指令执行期间触发的，CPU 会立即停止当前指令的执行并响应，而不是等到指令末尾
B 选项错误：一条指令（特别是在 CISC 架构下）可能会访问多个内存地址，完全可能在执行这一条指令期间引发多次缺页中断
D 选项错误：缺页中断响应后，硬件保存断点，随后的页面调入、置换、修改页表等复杂逻辑完全是由操作系统内核的缺页中断处理程序（软件）完成的
C 选项正确：修改位为 0 说明页面自调入内存后未被修改，磁盘上的副本仍然是最新的，直接丢弃内存副本即可，避免了昂贵的磁盘写回操作

7. 【考点：页框分配与回收】现代操作系统为管理虚拟内存，会采用不同的物理页框分配和置换策略。在“可变分配，全局置换”策略的运行机制中，下列说法最准确的是（ ）

A. 当进程发生缺页时，操作系统只能从该进程自身已占用的物理页框池中挑选一页进行淘汰置换，以保证该进程拥有的页框总数恒定不变
B. 系统必须在运行前精确计算出每个进程所需的物理页框上限，并在进程运行的全生命周期中严格维持这一物理配额
C. 只要系统为某进程分配的物理页框数量大于该进程理论上的工作集（Working Set）大小，该进程在运行时的缺页率就一定会严格降为零
D. 操作系统通常维护一个全局的空闲物理页框队列，发生缺页时优先从该队列中取空闲页分配给进程；仅当空闲队列枯竭或低于阈值时，才会触发全局页面置换算法剥夺其他进程的页框

【答案】：D
【解析】：
A 选项描述的是“固定分配，局部置换”或者“可变分配，局部置换”的置换逻辑，全局置换允许夺取别的进程的页框
B 选项描述的是“固定分配”的特征
C 选项错误：工作集只能防止“抖动（Thrashing）”，保持一个较低的可接受的缺页率。程序初次访问数据时的冷启动缺页（强制性缺页）是无论分配多少物理内存都无法避免的，缺页率不可能降为绝对的零
D 选项正确：这是“可变分配全局置换”在现代操作系统（如 Linux）中的标准实现方式，利用空闲链表缓冲缺页冲击

8. 【考点：页置换算法】页面置换算法的优劣直接决定了虚拟存储器的运行效率。关于各种经典页置换算法的设计特性与边界行为，下列推断正确的一项是（ ）

A. 最久未使用算法（LRU）因为性能出众且无需额外的硬件寄存器支持，是目前所有通用操作系统中处理缺页异常时最底层、最直接使用的代码级置换算法
B. 最佳置换算法（OPT）每次选择未来最长时间内不再被访问的页面进行淘汰，这种前瞻性使其成为当前公有云服务器中最广泛部署的在线置换算法
C. 在简单的时钟置换算法（Clock / NRU）中，当扫描指针遇到一个访问位（Use Bit）为 1 的页面时，会立即将该页面踢出内存，并将访问位清零后移动指针
D. 在所有已知的页面置换算法中，仅有先进先出算法（FIFO）存在一个著名的反常特性：即当系统分配给进程的物理页框总数增加时，其缺页故障次数反而可能会上升（Belady 异常）

【答案】：D
【解析】：
A 选项错误：LRU 算法需要极大的硬件开销（如移位寄存器或栈硬件机制），软件纯手工维护代价过高，因此实际操作系统使用的是其近似算法（如 Clock 算法）
B 选项错误：OPT 算法需要预知未来程序的访问流，这在物理现实中是不可能实现的，只能作为评估其他算法的理论上限指标
C 选项错误：Clock 算法遇到访问位为 1 的页面时，会给予该页一次“宽恕”（不清出内存），只是将其访问位清 0，然后指针向下移动检查下一个页面
D 选项正确：Belady 异常是 FIFO 算法独有的反常现象（由于它没有考虑页面的局部性访问历史）。LRU 和 OPT 均属于堆栈类算法，绝对不会出现 Belady 异常

9. 【考点：内存映射文件】内存映射文件（mmap）是现代操作系统提供的高级文件 I/O 与内存管理接口。以下关于该机制底层原理的叙述中，错误的是（ ）

   A. 内存映射技术将磁盘上的文件数据直接逻辑映射到进程的虚拟地址空间中，进程可以通过普通的指针操作（访存指令）来读写文件，完全避免了传统 `read()` / `write()` 系统调用的上下文切换开销
   B. 当多个无关的用户进程通过映射同一个物理磁盘文件来实现共享内存通信时，该文件数据在操作系统的物理内存层面上通常仅保留唯一的一份数据页框副本
   C. 当用户进程向内存映射区域的数据进行修改写入后，为了保证数据的绝对一致性，底层硬件会自动触发强制中断，并在当前时钟周期内立刻将新数据同步刷写回物理机械磁盘中
   D. 映射建立之初，系统通常只是在进程的页表中分配了逻辑地址空间，磁盘文件数据并未实际加载入物理内存；数据的真实载入是由后续访问引发的缺页中断驱动的

   【答案】：C
   【解析】：
   C 选项说法错误。内存映射文件的写入操作修改的是内存中的页框（将其标记为脏页 Dirty Page）。操作系统内核的后台线程（如 pdflush）会在未来的某个合适时机（或者用户显式调用 `msync` 时）将脏页异步批量延迟写回磁盘。硬件不可能也不允许在每次内存写入时强制同步写磁盘，否则系统的 I/O 性能将跌至冰点。其他选项均正确描述了 mmap 的优秀特性

10. 【考点：虚拟存储性能】当计算机系统进入严重的“抖动（Thrashing）”状态时，表现为系统响应极度迟缓甚至停滞。分析造成这种现象的根源及有效的系统改进应对方式，下列说法正确的是（ ）

    A. 系统发生抖动时，操作系统的 CPU 利用率往往会飙升至近乎 100%，因为处理器正在全力以赴地执行用户态程序的繁重计算任务
    B. 为彻底根治内存抖动并消除缺页中断的时间延迟，应由内核将系统的页面大小（Page Size）调整得越小越好，以实现更精准的数据载入
    C. 操作系统的进程调度模块监控到 CPU 利用率过低时，总是可以通过盲目增加系统中的多道程序度（调入更多新进程）来恢复系统吞吐量
    D. 工作集（Working Set）模型是缓解抖动的重要理论依据。只要系统能保证分配给各个活跃进程的驻留集物理页框总数略大于它们各自的当前工作集总和，就能有效避免系统陷入抖动崩溃

    【答案】：D
    【解析】：
    A 选项错误：抖动发生时，由于频繁缺页，所有进程都在等待磁盘 I/O，此时 CPU 处于极度空闲状态，CPU 有效利用率极低
    B 选项错误：页面过小会导致每个进程产生极其庞大的页表，占用大量内存；且会导致更频繁的缺页中断和磁盘碎片化传输，加剧性能恶化
    C 选项错误：如果在抖动状态下（CPU 利用率低是因为都在等 I/O），调度程序继续盲目增加多道程序度（引入新进程抢夺本已枯竭的物理内存），只会导致缺页情况雪上加霜，形成恶性循环。正确的做法是降低多道程序度，挂起部分进程
    D 选项正确：工作集模型指出，只有保证进程的工作集完全装入物理内存，进程才能平稳运行。因此依据工作集大小动态分配物理驻留集，是预防和控制系统抖动的最核心、最有效的改进方式之一

11. 【考点：页式管理】现代浏览器引擎或大型 C/C++ 应用在架构设计时，往往需要利用操作系统底层的内存分配与保护机制来隔离恶意脚本。关于连续分配、页式管理与段式管理的工程特征，下列说法完全正确的是（ ）

    A. 在纯段式存储管理中，由于段的长度是可变的，当系统长时间运行并频繁加载/卸载模块时，物理内存中会产生大量无法被直接利用的外部碎片（External Fragmentation），通常需要通过内存紧凑（Compaction）技术来解决
    B. 页式管理的核心优势在于它天然按逻辑功能模块（如主程序、数据区、共享库）划分虚拟内存，因此比段式管理更容易实现两个进程间对动态链接库（DLL/.so）的代码共享
    C. 无论是段式管理还是页式管理，为了保证系统的安全性，硬件 MMU 在进行地址转换时，只会检查虚拟地址是否超出了物理内存的绝对上限，而不检查页表或段表中的读/写/执行（R/W/X）权限控制位
    D. 在段页式管理（Segmentation with Paging）中，用户的虚拟地址空间首先被划分为大小相等的页，然后再将这些物理页组合成不同长度的段，从而完美结合了二者的优点

    【答案】： A
    【解析】：
    A 选项完全正确：纯段式管理的致命弱点就是外部碎片。因为每次分配的段大小不一，进程撤销后会在内存中留下许多不连续的小空闲块。要利用这些碎片，操作系统只能挂起进程，进行内存拷贝以拼接空闲区（即内存紧凑），这会带来巨大的性能开销
    B 选项错误：描述反了。段式管理才是按用户的逻辑实体（如函数、数组）划分的，因此更容易实现共享和保护。页式管理是对物理内存的机械切割（固定大小如 4KB），对用户完全透明，一个逻辑函数可能被强行截断在两个不同的物理页中，导致共享极为困难
    C 选项错误：现代内存保护机制的核心恰恰是依赖页表项（PTE）或段表项中的权限控制位（如 R/W/X 和特权级）。如果用户态程序试图向只读的数据页写入数据，MMU 会立刻触发段错误（Segmentation Fault / 越权异常）
    D 选项错误：段页式管理的逻辑顺序是“先分段，再分页”。即用户的逻辑地址空间先划分为多个有意义的段，然后将每个段内部再划分为大小相等的页。选项中“先分页再合成段”的表述是根本错误的

12. 【考点：请求页式管理】在开发如 Kafka 或 RocketMQ 等高吞吐量分布式消息队列时，为了极大提升海量日志文件的读写速度，工程上摒弃了传统的 `read/write` 系统调用，转而大量使用内存映射文件（Memory-Mapped Files，如 `mmap`）。关于 `mmap` 的底层运行机制，下列说法完全正确的是（ ）

    A. 当应用程序调用 `mmap()` 函数成功返回时，操作系统已经将磁盘上的该文件一次性完整加载到了物理内存中，从而保证了后续访存绝对不会发生缺页阻塞
    B. `mmap` 允许多个不同的进程将磁盘上的同一个文件映射到各自虚拟地址空间的不同区域，在物理内存层面，这些进程实际上共享着同一份页面缓存（Page Cache），从而实现了极速的 IPC（进程间通信）
    C. 在 32 位操作系统中，无论物理内存有多大，利用 `mmap` 机制单个进程可以轻松地一次性映射并处理一个大小为 10GB 的超大日志文件，因为映射操作不占用真实的物理内存
    D. 使用 `mmap` 修改了内存中的数据后，数据会绕过操作系统的页缓存（Page Cache）机制，直接通过 CPU 的前端总线同步烧录写入磁盘扇区

    【答案】： B
    【解析】：
    B 选项完全正确：这正是 `mmap` 被誉为零拷贝/极速 IPC 核心技术的本质。不同进程的页表项可以指向相同的物理内存页（即内核的 Page Cache）。进程 A 往映射的内存区域写入数据，进程 B 立刻就能在自己的虚拟地址中读到，中间没有任何内核态到用户态的数据拷贝开销
    A 选项错误：`mmap` 遵循请求调页（Demand Paging）机制。`mmap()` 函数调用时只是在进程的虚拟地址空间中分配了一段 VMA（虚拟内存区域），并未真正分配物理内存和读取磁盘。只有当 CPU 真正访问该地址引发缺页异常（Page Fault）时，操作系统才会将磁盘数据换入物理内存
    C 选项错误：`mmap` 必须占用进程的连续虚拟地址空间。32 位系统的总虚拟地址空间仅有 4GB（通常用户态只有 3GB）。因此，在 32 位系统上绝对不可能一次性 `mmap` 映射一个 10GB

的大文件，会直接报 `ENOMEM`（内存不足）错误
D 选项错误：`mmap` 修改的数据依然驻留在主存的 Page Cache 中，此时这些页变为“脏页（Dirty Pages）”。它们是由操作系统的后台刷盘内核线程（如 `pdflush`）异步写入磁盘的，或者由用户主动调用 `msync()` 强制刷盘，绝不可能“直接绕过缓存同步烧录磁盘”

13. 【考点：页置换算法】内存数据库（如 Redis）或关系型数据库（如 MySQL InnoDB）的缓冲池管理在理论上与操作系统的请求页式管理一脉相承。在物理内存页框（Frame）耗尽时，系统需要选择牺牲页。关于页置换算法的理论与工程实践，下列说法完全正确的是（ ）

    A. 先进先出（FIFO）置换算法由于实现极其简单，在现代操作系统中被广泛用作默认置换算法。此外，FIFO 算法天然保证了系统分配给进程的物理页框数越多，缺页率就一定越低
    B. 最佳置换算法（OPT）每次淘汰的是以后永不使用或最长时间内不再被访问的页面。由于该算法可以由硬件的“未来预测分支器”精确实现，因此在实时操作系统中应用广泛
    C. 纯正的最近最久未使用（LRU）算法在命中率上表现优异，但在操作系统内核中由于需要极其频繁地维护链表或更新时间戳，硬件开销和软件开销过大，因此 Linux 等实际系统往往采用 Clock（或 NRU）算法作为其近似实现
    D. 简单的时钟（Clock）置换算法只使用了一个访问位（Reference Bit）。在遍历寻找牺牲页时，只要遇到访问位为 1 的页面，系统会立即将其淘汰并装入新页

    【答案】： C
    【解析】：
    C 选项完全正确：真正的 LRU 要求在每一次内存访问（纳秒级）时都更新硬件时间戳或移动双向链表指针。这在纯软件层面会造成灾难性的性能损耗，在硬件层面成本也极高。因此，现代操作系统内核都采用基于访问位（Reference Bit）和修改位（Dirty Bit）的 Clock（时钟）算法或 NRU（最近未用）算法作为 LRU 的工程折中近似
    A 选项错误：FIFO 算法不仅性能差，更致命的是它存在 Belady 异常（分配的物理块数增加，缺页率反而可能上升）。由于其表现糟糕，现代 OS 根本不会用纯 FIFO 作为全局页置换算法
    B 选项错误：OPT（最佳置换算法）需要“预知未来”，这在物理上和逻辑上都是不可能实现的（图灵停机问题）。OPT 仅仅是一个理论上的性能上限，只能用于事后分析和对比，不可能在任何实际系统中实现
    D 选项错误：简单时钟（Clock）算法的逻辑是“给一次机会”。遇到访问位为 1 的页面时，会将其置为 0（而不是淘汰），指针继续向下扫描；只有遇到访问位为 0 的页面时，才会将其作为牺牲页淘汰

14. 【考点：页框分配与回收】在公有云容器（Docker/Kubernetes）环境中，节点上通常运行着负载特征迥异的多个进程。为了防止相互干扰，操作系统对虚拟存储器的页框分配策略及性能监控有着严格的要求。下列关于页框分配、回收机制及系统抖动的论述，完全正确的是（ ）

    A. 系统抖动（Thrashing）发生的核心标志是：CPU 的利用率急剧飙升至 100%，且系统的整体吞吐量达到极限上限
    B. 如果采用“全局置换（Global Replacement）”策略，当进程 A 发生缺页且自身配额用尽时，它可以剥夺并置换进程 B 的物理页框。这可能会导致原本运行良好的进程 B 无辜陷入缺页频繁的状态
    C. 根据工作集（Working Set）模型，系统应当实时监控所有进程，只要保证系统当前空闲的总物理页框数大于某一个进程的工作集大小，就可以彻底消除整个系统的抖动现象
    D. 在请求分页系统中，“固定分配、局部置换”策略意味着每个进程在运行期间，其持有的物理页框总数会根据它缺页率的动态高低而被操作系统频繁地增加或减少

    【答案】： B
    【解析】：
    B 选项完全正确：全局置换策略允许系统在整个物理内存中寻找牺牲页。其缺点就在于缺乏硬性隔离——进程 A 的大量内存请求会不断抢占进程 B 的页面，导致进程 B 缺页率上升。在云原生环境中，通常需要结合 Cgroups 等机制施加资源硬限制（即倾向于局部置换特征）来解决此类“吵闹的邻居（Noisy Neighbor）”问题
    A 选项错误：抖动发生的本质是系统将绝大部分时间花在磁盘的换入换出上（I/O 阻塞）。因此，抖动发生时，CPU 利用率会急剧下降（甚至跌至近乎 0%），而不是飙升，此时系统吞吐量极度恶

化

C 选项错误：工作集理论指出，要防止整个系统抖动，必须保证系统当前可提供的总物理页框数大于或等于所有并发进程的工作集大小之和（$\sum W S_i \le M$）。仅仅大于"某一个"进程的工作集是远远不够的

D 选项错误：描述自相矛盾。"固定分配"意味着进程在创建时被分配了绝对固定数量的物理页框，在整个生命周期内不能增加也不能减少。选项中描述的"根据缺页率动态增加或减少"属于可变分配策略

15. 【考点：地址变换】当 C++ 程序员在代码中解引用一个指针（如 `int val = *ptr;`）时，CPU 内部会经历极其复杂的硬件级地址变换过程。在采用两级页表及 TLB（快表）的现代操作系统中，关于地址变换逻辑的详细物理过程，下列描述完全正确的是（ ）

A. CPU 发出的虚拟地址 `ptr` 被分割为两部分：页号和页内偏移量。硬件 MMU 会直接将该虚拟地址送到内存的数据总线上进行物理寻址

B. 如果该虚拟地址在 TLB 中未命中（TLB Miss），则必然表明该页面不在物理内存中，MMU 会立刻触发缺页异常（Page Fault）中断，交由内核处理

C. 在最坏的情况下（TLB 未命中且页面在内存中），CPU 获取这一级数据，需要访问三次主存：第一次访问页目录表，第二次访问页表，第三次访问真正的物理目标地址

D. 只要 TLB 命中（TLB Hit），硬件会直接根据 TLB 中的物理块号计算出绝对物理地址，但随后仍必须去主存中的页表校验该页框的读写控制权限

【答案】： C

【解析】：

C 选项完全正确：在两级页表结构中，TLB 未命中的正常地址转换过程为：1. 访问内存中的页目录表（Page Directory），获取页表物理地址；2. 访问内存中的页表（Page Table），获取目标物理页框号；3. 拼接页内偏移量，最后一次访问目标物理地址获取数据。因此，获取一次数据总共引发了三次访存开销

A 选项错误：虚拟地址是绝对不能直接放到内存数据/地址总线上的。必须由 CPU 内部的 MMU（内存管理单元）通过页表映射转化为真实物理地址后，物理地址才能上总线

B 选项错误：TLB 仅仅是页表的缓存。TLB Miss 仅仅意味着映射关系不在缓存中。MMU 接着会去主存中遍历页表（Page Walk）。如果在主存页表中发现该页项的有效位（Present Bit）为 1，则直接算出物理地址并更新 TLB，不会触发任何缺页异常；只有当主存页表中的有效位为 0 时，才会触发 Page Fault

D 选项错误：TLB 不仅缓存了虚拟页号到物理块号的映射，同时也缓存了该页面的所有访问权限位（如 R/W、特权级）。只要 TLB 命中，硬件会直接在 TLB 内部完成权限校验并合成物理地址，绝对不需要再去主存的页表中验证权限，否则就失去了 TLB 加速的意义

16. **72．A**

伙伴算法在回收一个大小为 2^i 的空闲分区时，会检查其**伙伴分区**（地址相邻、大小同为 2^i 的分区）是否也空闲。若是，则将两者**合并**为一个大小为 2^{i+1} 的空闲分区；随后，继续检查新生成的 2^{i+1} 分区是否存在同大小的伙伴，若有，则**再次合并**，以此类推。因此，伙伴算法在一次回收过程中可能触发多次合并，但每次合并仅发生在两个大小相等且互为伙伴的空闲分区之间。

17. **59．B**

系统为进程 P 分配 3 个页框，记为 A、B、C，初始时页面 0、1、2 已在内存中。采用固定分配局部置换的 LRU 算法，访问页面序列时，各页框内容及缺页情况如下表所示。

	初始	#0	#1	#2	#0	#5	#1	#4	#3	#0	#2	#3	2	0
页框 A	#0	#0	#0	#0	#0	#0	#0	**#4**	#4	#4	**#2**	#2		
页框 B	#1	#1	#1	#1	#1	**#5**	#5	#5	**#3**	#3	#3	#3		
页框 C	#2	#2	#2	#2	#2	#2	**#1**	#1	#1	**#0**	#0	#0		
是否缺页						是	是	是	是	是	是			

初始状态：页框 A、B、C 分别存放#0、#1、#2。第 5 次访问#5：不在内存，缺页，此时最近使用顺序为#0→#2→#1（#1 为最久未用），淘汰最久未使用的#1，换入#5。第 6 次访问#1：不在内存，缺页，淘汰最久未用的#2，换入#1。第 7 次访问#4：缺页，淘汰#0，换入#4。第 8 次访问#3：缺页，淘汰#5，换入#3。第 9 次访问#0：缺页，淘汰#1，换入#0。第 10 次访问#2：缺页，淘汰#4，换入#2。后续访问#3、#2、#0 均命中。共发生 6 次缺页异常。

18. **60．D**

在页式系统中，进程正常运行所需的**最少页框数**并不由代码段长度、虚拟地址空间大小或物理内存容量决定，而取决于**单条指令在取指和执行过程中可能同时访问的最大页面数**。该数量由指令系统支持的寻址方式直接决定：取指阶段至少需访问存放指令的页面；执行阶段则因寻址方式不同而异。寄存器寻址无须访问内存，直接寻址需访问一个数据页面，而间接寻址可能需先后访问两个页面（分别存放形式地址和实际操作数）。若一条指令最多涉及 k 个不同的页面，则系统至少需要为其分配 k 个页框，否则将因频繁缺页而导致颠簸，甚至无法完成该指令的执行。因此，确定最少页框数的关键因素是指令系统的寻址方式。

19. **61．A**

内存映射文件机制**将文件内容映射到进程的虚拟地址空间**，使进程能够像访问内存一样直接读/写文件数据，而底层 I/O（如页面的按需加载与换出）由操作系统自动处理，从而简化文件操作。当多个进程映射同一文件时，它们通过共享内核页缓存读/写相同的数据，从而**高效实现进程间通信**。需要明确的是，该机制本身仅建立文件与进程虚拟地址空间之间的关联，并不直接实现“页到磁盘块的映射”，这一映射由操作系统的虚拟内存管理系统（如页表、缺页中断机制）统一维护，属于通用虚拟存储机制的功能。此外，映射目标始终是进程的虚拟地址空间，物理地址由操作系统动态分配并对进程完全透明，进程无法感知或直接访问。因此，只有说法 I 和 III 准确描述了内存映射文件机制直接提供的功能。

20. **11．【解答】**

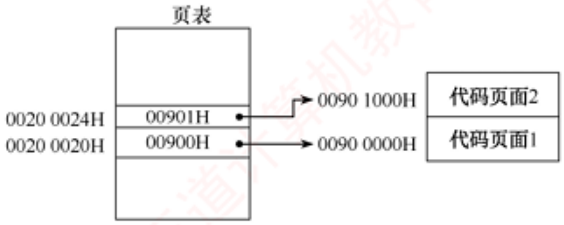
1）因为主存按字节编址，页内偏移量是 12 位，所以页大小为 2^{12}B = 4KB。

页表项数为 2^{32} /4K = 2^{20}，因此该一级页表最大为 2^{20}×4B = 4MB。

2）页目录号可表示为(((unsigned int)(LA))>>22) & 0x3FF。这里采用的方法是逻辑右移 22 位，再和 3FF（10 个 1）进行逻辑与运算，得到 10 位的页目录号。这种方法虽然效率较高，但比较难想到，采用 LA/2^{22} 的写法来取高 10 位的页目录号也是可以的。

页表索引可表示为(((unsigned int)(LA))>>12) & 0x3FF。这里也可采用(LA/2^{12})%2^{10} 的方法来获取中间 10 位的页表索引号。

3）代码页面 1 的逻辑地址为 0000 8000H，表明其位于第 8 个页处，对应页表中的第 8 个页表项，所以第 8 个页表项的物理地址 = 页表始址 + 8×页表项的字节数 = 0020 0000H + 8×4 = 0020 0020H。由此可得如下图所示的答案。



21. **15．【解答】**

1）根据页式管理的工作原理，应先考虑页面大小，以便将页号和页内位移分解出来。页面大小为 4KB，即 2^{12}，得到页内位移占虚地址的低 12 位，页号占剩余高位。可得三个虚地址的页号 P 如下（十六进制的一位数字转换成二进制的 4 位数字，因此十六进制的低三位正好为页内位移，最高位为页号）：

2362H：P = 2，访问快表 10ns，因初始为空，访问页表 100ns 得到页框号，合成物理地址后访问主存 100ns，共计 10ns + 100ns + 100ns = 210ns。

1565H：P = 1，访问快表 10ns，落空，访问页表 100ns 落空，进行缺页中断处理 10^8ns，访问快表 10ns，合成物理地址后访问主存 100ns，共计 10ns + 100ns + 10^8ns + 10ns + 100ns = 100000220ns。

25A5H：P = 2，访问快表，因第一次访问已将该页号放入快表，因此花费 10ns 便可合成物理地址，访问主存 100ns，共计 10ns + 100ns = 110ns。

2）当访问虚地址 1565H 时，产生缺页中断，合法驻留集为 2，必须从页表中淘汰一个页面，根据题目的置换算法，应淘汰 0 号页面，因此 1565H 的对应页框号为 101H。由此可得 1565H 的物理地址为 101565H。

22.

16.【解答】

1）因为该计算机的逻辑地址空间和物理地址空间均为 64KB = 2^{16}B，按字节编址，且页的大小为 1K = 2^{10}，所以逻辑地址和物理地址的地址格式均为

页号/页框号（6 位）	页内偏移量（10 位）

17CAH = 0001 0111 1100 1010B，可知该逻辑地址的页号为 000101B = 5。

2）采用 FIFO 算法，与最早调入的页面即 0 号页面置换，其所在的页框号为 7，于是对应的物理地址为 0001 1111 1100 1010B = 1FCAH。

3）采用 CLOCK 算法，首先从当前位置（2 号页框）开始顺时针寻找访问位为 0 的页面，当指针指向的页面的访问位为 1 时，就将该访问位清零，指针遍历一周后，回到 2 号页框，此时 2 号页框的访问位为 0，置换该页框的页面，于是对应的物理地址为 0000 1011 1100 1010B = 0BCAH。

23. **17.【解答】**

1）页框号为 21。因为起始驻留集为空，而 0 页对应的页框为空闲链表中的第三个空闲页框（21），其对应的页框号为 21。

2）页框号为 32。理由：因 11 > 10，因此发生第三轮扫描，页号为 1 的页框在第二轮已处于空闲页框链表中，此刻该页又被重新访问，因此应被重新放回驻留集中，其页框号为 32。

3）页框号为 41。理由：因为第 2 页从来没有被访问过，它不在驻留集中，因此从空闲页框链表中取出链表头的页框 41，页框号为 41。

4）合适。理由：程序的时间局部性越好，从空闲页框链表中重新取回的机会越大，该策略的优势越明显。

24. **18.【解答】**

1）页和页框大小均为 4KB。进程的虚拟地址空间大小为 $2^{32}/2^{12} = 2^{20}$ 页。

2）$(2^{10}\times4)/2^{12}$（页目录所占页数）+ $(2^{20}\times4)/2^{12}$（页表所占页数）= 1025 页。页目录占用的物理页面数 =（页目录项数×页目录项大小）/物理页面大小 = $(2^{10}\times4)/2^{12}$ = 1 页。虚拟地址空间中的高 20 位表示有多少个页面，所以进程的页面数为 2^{20}，因此页表总大小为 $2^{20}\times4$B，存放页表需要 $(2^{20}\times4)/2^{12}$ = 1024 页。因此，页目录和页表共占 1 + 1024 = 1025 页。

3）需要访问一个二级页表。因为虚拟地址 0100 0000H 和 0111 2048H 的最高 10 位的值都是 4，访问的是同一个二级页表。

25. **19.【解答】**

1）函数 f1 的代码段中，所有指令的虚拟地址的高 20 位相同，因此 f1 的机器指令代码在同一页中，仅占用 1 页。页目录号用于寻找页目录的表项，该表项包含页表的位置。页表索引用于寻找页表的表项，该表项包含页的位置。

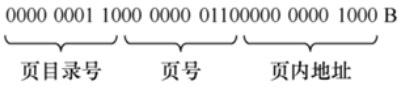
2）push ebp 指令的虚拟地址的最高 10 位（页目录号）为 00 0000 0001，中间 10 位（页表索引）为 00 0000 0001，所以取该指令时访问了页目录的 1 号表项，在对应的页表中访问了 1 号表项。

3）在执行 scanf()的过程中，进程 P 因等待输入而从执行态变为阻塞态。输入结束时，P 被中断处理程序唤醒，变为就绪态。P 被调度程序调度，变为运行态。CPU 状态会从用户态变为内核态。

26.

20.【解答】

1）由图可知，地址总长度为 32 位，高 20 位为虚页号，低 12 位为页内地址，且虚页号高 10 位为页目录号，低 10 位为页号。展开成二进制表示为



因此十六进制表示为 0180 6008H。

2）PDBR 为页目录基址地址寄存器（Page-Directory Base Register），其存储页目录表物理内存基地址。进程切换时，PDBR 的内容会变化；同一进程的线程切换时，PDBR 的内容不会变化。每个进程的地址空间、页目录和 PDBR 的内容存在一一对应的关系。进程切换时，地址空间发生了变化，对应的页目录及其始址也相应变化，因此需要用进程切换后当前进程的页目录始址刷新 PDBR。同一进程中的线程共享该进程的地址空间，其线程发生切换时，地址空间不变，线程使用的页目录不变，因此 PDBR 的内容也不变。

3）改进型 CLOCK 算法需要用到使用位和修改位，所以需要设置访问字段（使用位）和修改字段（脏位）。

27. **21.【解答】**

1）

① 页面大小 $=2^{12}$B = 4096B = 4KB。每个数组元素 4B，每个页面可以存放 4KB/4B = 1024 个数组元素，正好是数组的一行，数组 a 按行优先方式存放。1080 0000H 的虚页号为 10800H，因此 a[0]行存放在虚页号为 10800H 的页面中，a[1]行存放在页号为 10801H 的页面中。a[1][2]的虚拟地址为 10801 000H + 4×2 = 10801 008H。

② 转换为二进制 0001000010 0000000001 000000001000，根据虚拟地址结构可知，对应的页目录号为 042H，页号为 001H。

③ 进程的页目录表始址为 0020 1000H，每个页目录项长 4B，因此 042H 号页目录项的物理地址是 0020 1000H + 4×42H = 0020 1108H。

④ 页目录项存放的页框号为 00301H，二级页表的始址为 00301 000H，因此 a[1][2]所在页的页号为 001H，每个页表项 4B，因此对应的页表项物理地址是 00301 000H + 001H×4 = 00301 004H。

2）根据数组的随机存取特点，数组 a 在虚拟地址空间中所占的区域必须连续，因为数组 a 不止占用一页，相邻逻辑页在物理上不一定相邻，所以数组 a 在物理地址空间中所占的区域可以不连续。

3）由 1）可知每个页面正好可以存放一整行的数组元素，“按行优先方式存放”意味着数组的同一行的所有元素都存放在同一个页面中，同一列的各个元素都存放在不同的页面中，因此数组 a 按行遍历的局部性较好。

28. **22.【解答】**

1）页表项的虚拟地址为 B8C0 0000H + 48H << 2 = B8C0 0120H。页表项的物理地址为 6540 0000H + 48H << 2 = 6540 0120H。相应页表项中的页框号为 BAB4 5678H >> 22 = 2EAH。

2）进程 P 的页表所在页的页号为 B8C0 0000H >> 22 = 2E3H。页表项的虚拟地址为 B8C0 0000H + 2E3H << 2 = B8C0 0B8CH。页表项中的页框号为 6540 0000H >> 22 = 195H。

注意，符号“<<”表示左移操作，符号“>>”表示右移操作。将二进制数左移 1 位，与“乘以 2”的效果等价。页表项中的页框号为物理地址的前 10 位可以通过右移 22 位得到。

29. 【考点：内存映射文件】某高性能系统采用请求分页存储管理及多级页表机制，且硬件配备了 TLB 与 Data Cache。现有一进程利用内存映射文件（Memory-Mapped Files）技术，将一个体积庞大的磁盘文件映射到自身的逻辑地址空间中。关于该进程后续运行及系统底层的响应机制，下列说法中错误的是（ ）

A. 在建立内存映射初期，操作系统仅在该进程的逻辑地址空间中分配相应的虚拟内存区域（VMA），并未实际分配物理页框和拷贝数据，因此首次对该区域访存必定触发缺页异常

B. 缺页异常是由硬件 MMU 在地址转换过程中发现页表项有效位（Present Bit）为 0 而直接触发的内中断（Fault），随后 CPU 切换至内核态并执行缺页异常处理程序

C. 缺页异常处理程序完成物理页框分配与磁盘 I/O 后，会执行中断返回指令（如 `iret`）重新执行被中断的访存指令；为了提升性能，硬件 MMU 会直接利用异常处理期间缓存在内部控制寄存器中的物理页号进行访存，跳过 TLB 与页表的查询流程

D. 若系统内存资源紧张触发了页框回收机制，且恰好选中了该内存映射文件对应的一个“脏页（Dirty Page）”作为淘汰页，系统无需将其换出至交换区（Swap），而是直接将其数据写回映射的底层磁盘文件

【答案】： C

【解析】：

A 项正确：这是内存映射文件（mmap）的核心机制（延迟加载/按需调页）。系统调用建立映射时只修改进程的虚拟地址空间分布，并不分配实际的物理内存。真正的数据装载推迟到 CPU 首次尝试读写该虚拟地址引发缺页异常时进行

B 项正确：缺页异常属于“故障（Fault）”类的内部异常，它完全由硬件 MMU 在地址翻译阶段自发检测（有效位为0）并触发。操作系统被动接管并完成从用户态到内核态的上下文切换以分配页面

C 项错误：缺页异常属于 Fault 类型的异常，其中断返回的特点是重新从头执行引发异常的那条指令。由于是彻底重新执行，硬件 MMU 并没有所谓的“利用内部控制寄存器直接访存”的捷径机制。MMU 必须重新进行完整的逻辑地址到物理地址的翻译（通常此时 TLB 依然未命中，需硬件自动遍历位于内存中的页表结构，命中有效位已被 OS 置为 1 的页表项，取出物理页号，随后更新 TLB 并完成实际访存）

D 项正确：此处深刻考察了页框分配与回收策略。普通的匿名页（如堆、栈数据）若被修改过，淘汰时必须写回外存的交换区（Swap）。但内存映射文件页背后有具体的磁盘文件实体，其脏数据会被直接同步（写回）至原文件对应的磁盘块中，这是内存映射文件能高效修改文件内容的核心原理

30. 【考点：段页式管理地址结构】虚拟存储管理中，地址映射结构、页面分配策略及系统负载均会显著影响系统的整体性能。下列关于虚拟存储器架构设计及性能演变的说法中，错误的是（ ）

A. 在段页式管理系统中，用户编程时提供的是由“段号”和“段内偏移量”构成的二维逻辑地址；而底层硬件 MMU 会自动将“段内偏移量”进一步拆分为页号和页内偏移量，这一页式管理过程对用户是完全透明的

B. 虚拟存储器的最大支持容量由计算机的逻辑地址总线位数（即 CPU 的寻址能力）唯一决定；而在实际运行中，其实际有效容量则是物理内存实际大小与磁盘交换区（Swap）大小之和

C. 当系统的并发进程数过多时，若各个进程的驻留集（Resident Set）大小均小于其工作集（Working Set），系统将陷入频繁调页的“抖动（Thrashing）”状态，此时 CPU 的有效利用率将急剧下降

D. 为了改善“抖动”导致系统性能断崖式下跌的现象，由于此时 CPU 大量时间处于等待磁盘 I/O 的空闲状态，操作系统应当自动增加多道程序度（引入更多新进程），以充分榨取 CPU 的空闲算力

【答案】： D

【解析】：

A 项正确：在段页式系统中，用户的视角依然是段式管理，地址空间具有二维性（需显式指明在哪个段以及段内的相对偏移）。但为了解决内存的外部碎片问题，操作系统将每一段进一步划分为固定大小的页。硬件 MMU 在执行地址转换时隐式切分偏移量，实现了面向物理的页式分配，这对上层程序员是不可见的

B 项正确：此为虚拟内存基本概念的核心考点。虚拟内存的最大容量受限于计算机体系结构（如 32 位系统逻辑地址空间为 4 GB）。其实际容量受物理内存容量和外存容量的限制，通常定义为 min(内存容量 + 外存交换区容量, 最大寻址空间)

C 项正确：这是“抖动”发生的根本原因。工作集是指进程在某段时间内频繁访问的页面集合。若操作系统分配给进程的物理页框（驻留集）不足以容纳整个工作集，就会导致刚换出的页面很快又被访问，从而引发高频缺页异常，系统算力被内耗在上下文切换和 I/O 等待上

D 项错误：当系统发生抖动时，CPU 利用率下降是因为 CPU 时间全被用于处理缺页中断（系统内核态开销），而不是因为系统真的没有任务可做。如果此时操作系统（早期的初级调度器极易犯此错误）观察到 CPU 利用率低，反而继续增加多道程序度，新加入的进程会争夺本就极其匮乏的物理页框，导致缺页率进一步飙升，引发系统彻底崩溃（雪崩效应）。正确的改进方式是降低多道程序度，挂起（Suspend）部分非关键进程并回收其页框，直至抖动消除